ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN – CƠ – TIN HỌC

BÁO CÁO CUỐI KỲ
Môn học: **TÍNH TOÁN SONG SONG**

PARALLEL PREFIX SUM USING OPENMP: IMPLEMENTATION AND PERFORMANCE EVALUATION

(Tính tổng tiền tố song song bằng OpenMP: cài đặt và đánh giá hiệu năng)

Giảng viên hướng dẫn: Nguyễn Hải Vinh
Mạc Đăng Trường

Nhóm sinh viên thực hiện:

STT	Họ và tên	Mã sinh viên
1	Bùi Xuân Chung	23001838
2	Lê Văn Cường	23001842
3	Đào Ngọc Cường	23001841

Hà Nội, tháng 12 năm 2026

LỜI CẢM ƠN

Nhóm chúng em xin gửi lời cảm ơn chân thành tới thầy **Nguyễn Hải Vinh** và thầy **Mạc Đăng Trường**, giảng viên học phần Tính toán song song, Trường Đại học Khoa học Tự nhiên – Đại học Quốc gia Hà Nội. Trong suốt học kỳ, các thầy đã truyền đạt cho chúng em những kiến thức nền tảng về kiến trúc máy tính song song, mô hình lập trình bộ nhớ chia sẻ và phương pháp đánh giá hiệu năng; đồng thời góp ý, định hướng để nhóm hoàn thành đề tài này.

Chúng em cũng xin cảm ơn các bạn trong lớp đã trao đổi, thảo luận và chia sẻ kinh nghiệm trong quá trình học tập. Do thời gian và kiến thức còn hạn chế, báo cáo khó tránh khỏi những thiếu sót; nhóm rất mong nhận được ý kiến đóng góp của thầy và các bạn để báo cáo được hoàn thiện hơn.

Hà Nội, tháng 12 năm 2026

Nhóm sinh viên thực hiện

Bùi Xuân Chung

Lê Văn Cường

Đào Ngọc Cường

TÓM TẮT

BẢNG PHÂN CÔNG CÔNG VIỆC

Thành viên	MSSV	Nhiệm vụ chính	Đóng góp
Bùi Xuân Chung	23001838	Tìm hiểu tài liệu (Blelloch 1990, GPU Gems 3); viết Chương 1 (Giới thiệu) và Chương 2 (Cơ sở lý thuyết: work–span, định luật Amdahl/Gustafson, mô hình roofline, OpenMP, kiến trúc CPU); trình bày phần mở đầu và cơ sở lý thuyết.	33%
Lê Văn Cường	23001842	Phân tích và cài đặt các thuật toán (Hillis–Steele, Blelloch, Blocked, Blocked-opt, `omp scan`); viết bộ kiểm thử tính đúng; viết Chương 3 (Thuật toán) và Chương 4 (Cài đặt); trình bày phần thuật toán và cài đặt.	34%
Đào Ngọc Cường	23001841	Thiết kế và thực hiện thực nghiệm (script đo, xử lý số liệu, vẽ biểu đồ); cài đặt ứng dụng stream compaction; viết Chương 5 (Thực nghiệm) và Chương 6 (Kết luận); trình bày phần kết quả và kết luận.	33%

Cả nhóm cùng tham gia thảo luận thiết kế, rà soát mã nguồn, đọc chéo báo cáo và chuẩn bị slide thuyết trình. Mã nguồn, số liệu thô và báo cáo được quản lý trên kho GitHub: `https://github.com/CuongMotNang/parallel-computing`.

MỤC LỤC

DANH SÁCH HÌNH VẼ

DANH SÁCH BẢNG

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ

Viết tắt / thuật ngữ	Ý nghĩa
API	Application Programming Interface – giao diện lập trình ứng dụng
AVX2	Advanced Vector Extensions 2 – tập lệnh SIMD 256 bit của x86
CPU / GPU	Bộ xử lý trung tâm / bộ xử lý đồ họa
CUDA	Compute Unified Device Architecture – nền tảng lập trình GPU của NVIDIA
EREW PRAM	Exclusive-Read Exclusive-Write Parallel Random Access Machine
E-core / P-core	Nhân tiết kiệm điện (Efficient) / nhân hiệu năng cao (Performance)
GB/s	Gigabyte trên giây (10^9 byte/s)
HT	Hyper-Threading – đa luồng đồng thời trên một nhân vật lý
LLC	Last-Level Cache – bộ nhớ đệm cấp cuối (L3)
MPI	Message Passing Interface
NUMA	Non-Uniform Memory Access
OpenMP	Open Multi-Processing – chuẩn lập trình song song bộ nhớ chia sẻ
PRAM	Parallel Random Access Machine – mô hình máy song song lý thuyết
SIMD	Single Instruction, Multiple Data
SMT	Simultaneous Multithreading
Scan	Phép tính tổng tiền tố (prefix sum)
Inclusive / exclusive scan	Tổng tiền tố bao gồm / không bao gồm phần tử hiện tại
Work (W)	Tổng số phép toán của thuật toán song song
Span / depth (D)	Độ dài đường găng – số bước tối thiểu khi có vô hạn bộ xử lý
Speedup (S_p)	Tỉ số T_{seq}/T_p
Efficiency (E_p)	Hiệu suất song song S_p/p

CHƯƠNG 1

GIỚI THIỆU

1.1 Bối cảnh

Trong gần hai thập kỷ trở lại đây, tốc độ xung nhịp của bộ xử lý gần như dừng lại ở mức 3–5 GHz do giới hạn về công suất tiêu thụ và tản nhiệt (hiện tượng *power wall*). Thay vì tăng tốc một nhân, các nhà sản xuất chuyển sang tích hợp nhiều nhân trên cùng một chip: ngay cả một máy tính xách tay phổ thông ngày nay cũng có từ 8 đến 16 luồng phần cứng, kèm theo các đơn vị tính toán vector (SIMD) rộng 256 hoặc 512 bit **hennessy2019**. Hệ quả là hiệu năng phần mềm không còn "tự động" tăng theo thế hệ phần cứng; muốn tận dụng tài nguyên tính toán, lập trình viên buộc phải *song song hóa* chương trình.

Song song hóa dễ nhất với các bài toán mà mỗi phần tử dữ liệu được xử lý độc lập (cộng hai vector, áp bộ lọc lên từng điểm ảnh) – loại bài toán thường gọi là *embarrassingly parallel*. Tuy nhiên, rất nhiều thuật toán quan trọng chứa *phụ thuộc dữ liệu* giữa các bước lặp, khiến việc chia việc cho nhiều luồng không còn hiển nhiên. Ví dụ điển hình, đồng thời là chủ đề của báo cáo này, là phép **tính tổng tiền tố** (*prefix sum*, hay *scan*):

$$y_i \;=\; x_0 \oplus x_1 \oplus \cdots \oplus x_i, \qquad i = 0, 1, \ldots, n-1, \tag{1.1}$$

trong đó $\oplus$ là phép toán hai ngôi có tính kết hợp (thường là phép cộng). Cài đặt tuần tự của (**??**) chỉ là vòng lặp $y_i = y_{i-1} \oplus x_i$; mỗi bước phụ thuộc trực tiếp vào bước liền trước. Thoạt nhìn, phép toán này có vẻ *bản chất tuần tự*.

Blelloch **blelloch1990** đã chỉ ra rằng điều đó không đúng: tổng tiền tố là một ví dụ điển hình về phép tính trông có vẻ tuần tự nhưng lại có thuật toán song song hiệu quả. Hơn nữa, scan được xem là một trong những *khối xây dựng* (building block) cơ bản nhất của thuật toán song song, có vai trò tương tự phép duyệt mảng trong thuật toán tuần tự. Rất nhiều bài toán – sắp xếp cơ số, sắp xếp nhanh, lọc phần tử (stream compaction), cấp phát bộ nhớ động, phân tích từ vựng, giải hệ truy hồi, dựng histogram, bảng tổng vùng trong xử lý ảnh – có thể quy về một hoặc vài lần gọi scan **blelloch1990**, **harris2007**, **sengupta2007**. Vì vậy, một cài đặt scan song song nhanh có ý nghĩa thực tiễn lớn: nó trực tiếp tăng tốc cả một lớp ứng dụng.

Phần lớn tài liệu kinh điển về scan song song tập trung vào mô hình lý thuyết PRAM **blelloch1990**, **ladner1980** hoặc vào GPU với hàng nghìn luồng **harris2007**, **merrill2016**. Trên CPU đa nhân – nơi số luồng chỉ khoảng chục, có hệ thống bộ nhớ đệm nhiều cấp và băng thông bộ nhớ hạn chế – các đánh đổi lại rất khác: một thuật toán "tối ưu" trên lý thuyết chưa chắc đã nhanh trong thực tế. Đây chính là động lực của đề tài: *cài đặt các thuật toán scan song song bằng OpenMP trên CPU đa nhân và đánh giá hiệu năng một cách có hệ thống*.

1.2 Phát biểu bài toán

Đầu vào. Mảng $x = [x_0, x_1, \ldots, x_{n-1}]$ gồm n số nguyên 64 bit lưu liên tiếp trong bộ nhớ, và số luồng p.

Đầu ra. Mảng y cùng kích thước thỏa mãn (**??**) với $\oplus$ là phép cộng (*inclusive scan*). Kết quả song song phải **trùng khớp tuyệt đối** với kết quả tuần tự.

Yêu cầu.

1. Cài đặt bằng C và OpenMP các thuật toán scan song song tiêu biểu: Hillis–Steele, Blelloch (up-sweep/down-sweep), scan chia khối và chỉ thị `scan` của OpenMP 5.0.
2. Kiểm chứng tính đúng trên nhiều kích thước dữ liệu và số luồng.
3. Đánh giá hiệu năng: thời gian chạy, speedup, hiệu suất, khả năng mở rộng (strong/weak scaling), băng thông bộ nhớ đạt được; phân tích nguyên nhân của các kết quả.

Việc chọn số nguyên 64 bit thay vì số thực là có chủ đích: phép cộng dấu phẩy động *không* có tính kết hợp tuyệt đối, nên thứ tự cộng khác nhau giữa các thuật toán sẽ cho sai khác nhỏ, gây khó cho kiểm thử. Với số nguyên, mọi sai khác dù chỉ một bit đều là lỗi.

1.3 Mục tiêu và phạm vi

- **Lý thuyết:** trình bày định nghĩa, các thuật toán scan song song kinh điển cùng phân tích theo mô hình work–span; làm rõ khái niệm *work-efficiency* và vì sao nó quan trọng khi số luồng nhỏ hơn nhiều so với kích thước dữ liệu.
- **Cài đặt:** xây dựng một thư viện nhỏ gồm sáu phiên bản scan với giao diện thống nhất, kèm bộ kiểm thử tự động và chương trình đo hiệu năng.
- **Thực nghiệm:** đo đạc trên một CPU lai (hybrid) hiện đại, đối chiếu với giới hạn băng thông bộ nhớ để trả lời câu hỏi: *scan song song trên CPU nhanh được đến đâu và bị giới hạn bởi điều gì?*
- **Ứng dụng:** minh họa dùng scan để song song hóa bài toán lọc phần tử.

Phạm vi. Đề tài giới hạn trong mô hình bộ nhớ chia sẻ trên một nút tính toán, sử dụng OpenMP. GPU (CUDA) và bộ nhớ phân tán (MPI) chỉ được đề cập ở mức tổng quan.

1.4 Đóng góp của nhóm

1. Cài đặt và kiểm thử sáu phiên bản scan bằng OpenMP, trong đó phiên bản tối ưu *Blocked-opt* kết hợp chia tile vừa bộ nhớ đệm L2, chống false sharing và vector hóa, giảm lưu lượng bộ nhớ từ 24 xuống 16 byte cho mỗi phần tử.
2. Xây dựng quy trình đo hiệu năng có thể tái lập (script tự động, nhiều lần lặp, lấy trung vị, ghi lại cấu hình máy) và công cụ sinh biểu đồ, bảng LaTeX tự động.
3. Phân tích kết quả dựa trên mô hình roofline: với dữ liệu lớn, scan là bài toán *giới hạn bởi băng thông bộ nhớ*, nên speedup bị chặn bởi tỉ số băng thông đa luồng/đơn luồng chứ không phải bởi số nhân.
4. Ghi nhận các vấn đề thực tế: thuật toán "đẹp" trên lý thuyết (Hillis–Steele, Blelloch) chậm hơn cả bản tuần tự trên CPU; chỉ thị `omp scan` của GCC lỗi khi vòng lặp rỗng; ảnh hưởng của kiến trúc lai P-core/E-core tới cân bằng tải.

1.5 Cấu trúc báo cáo

?? trình bày cơ sở lý thuyết: định nghĩa scan, mô hình work–span, các định luật giới hạn tăng tốc, mô hình roofline, chuẩn OpenMP và đặc điểm kiến trúc CPU. **??** trình bày chi tiết các thuật toán scan song song, phân tích độ phức tạp và chứng minh tính đúng. **??** mô tả cài đặt bằng OpenMP, kỹ thuật tối ưu và kiểm thử. **??** trình bày môi trường, phương pháp và kết quả thực nghiệm kèm phân tích. **??** tổng kết, nêu hạn chế và hướng phát triển.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

Chương này trình bày các khái niệm nền tảng được sử dụng trong toàn bộ báo cáo: định nghĩa hình thức của phép scan (**??**), mô hình phân tích thuật toán song song work–span (**??**), các định luật giới hạn tăng tốc và mô hình roofline (**??**), chuẩn lập trình OpenMP (**??**) và các đặc điểm kiến trúc CPU đa nhân ảnh hưởng trực tiếp tới hiệu năng (**??**).

2.1 Phép tính tổng tiền tố

Định nghĩa 2.1 (Inclusive scan)**.** Cho phép toán hai ngôi $\oplus$ có tính kết hợp trên tập X và dãy $x = [x_0, x_1, \ldots, x_{n-1}] \in X^n$. *Inclusive scan* của x là dãy

$$\text{scan}(x) = [\, x_0,\ x_0 \oplus x_1,\ \ldots,\ x_0 \oplus x_1 \oplus \cdots \oplus x_{n-1} \,].$$

Định nghĩa 2.2 (Exclusive scan / prescan)**.** Nếu $\oplus$ có phần tử đơn vị I, *exclusive scan* (Blelloch gọi là *prescan* **blelloch1990**) của x là

$$\text{prescan}(x) = [\, I,\ x_0,\ x_0 \oplus x_1,\ \ldots,\ x_0 \oplus \cdots \oplus x_{n-2} \,].$$

Ví dụ 2.1. Với $\oplus = +$, $I = 0$ và $x = [3, 1, 7, 0, 4, 1, 6, 3]$:

$$\text{scan}(x) = [3, 4, 11, 11, 15, 16, 22, 25], \qquad \text{prescan}(x) = [0, 3, 4, 11, 11, 15, 16, 22].$$

Hai dạng có thể chuyển đổi cho nhau với chi phí $\mathcal{O}(n)$ và hoàn toàn song song: $\text{scan}(x)_i = \text{prescan}(x)_i \oplus x_i$, và ngược lại $\text{prescan}(x)_i = \text{scan}(x)_{i-1}$ với $\text{prescan}(x)_0 = I$. Báo cáo này dùng inclusive scan làm đầu ra chuẩn cho mọi phiên bản; thuật toán Blelloch tự nhiên sinh ra exclusive scan và được chuyển đổi ở bước cuối.

Vai trò của tính kết hợp. Mọi thuật toán scan song song đều dựa trên việc *nhóm lại* các phép toán theo thứ tự khác với thứ tự tuần tự, ví dụ $(x_0 \oplus x_1) \oplus (x_2 \oplus x_3)$ thay vì $((x_0 \oplus x_1) \oplus x_2) \oplus x_3$. Điều này chỉ hợp lệ khi $\oplus$ kết hợp. Tính giao hoán *không* cần thiết: các thuật toán giữ nguyên thứ tự trái–phải của toán hạng. Do đó scan áp dụng được cho nhiều phép toán: cộng, nhân, max, min, AND/OR logic, nhân ma trận, hợp các hàm affine $f(y) = ay + b$ (dùng để giải hệ truy hồi tuyến tính bậc nhất) **blelloch1990**.

Vấn đề với số thực. Phép cộng dấu phẩy động IEEE 754 chỉ *gần* kết hợp: $(a + b) + c$ và $a + (b + c)$ có thể khác nhau ở bit cuối do làm tròn. Thuật toán song song vì vậy cho kết quả hơi khác bản tuần tự; sai số tích lũy của cách nhóm theo cây thậm chí thường *nhỏ hơn* cách cộng tuần tự. Báo cáo dùng số nguyên 64 bit để có thể so sánh chính xác từng bit.

Thuật toán tuần tự. Cài đặt tuần tự rất đơn giản (**??**): đúng $n - 1$ phép cộng, đây là số phép toán tối thiểu cần thiết, đồng thời là mốc để đánh giá các thuật toán song song. Tuy nhiên vòng lặp có *phụ thuộc mang qua vòng lặp* (loop-carried dependency): bước i cần kết quả bước $i - 1$, nên không thể đơn giản thêm `#pragma omp parallel for`.

Thuật toán 1. Inclusive scan tuần tự

Đầu vào: mảng $x[0..n-1]$

Đầu ra: mảng $y[0..n-1]$, $y_i = \sum_{j \le i} x_j$

```
s ← 0;
for i ← 0 to n − 1 do
    s ← s + x[i];
    y[i] ← s;
```

2.2 Mô hình phân tích thuật toán song song

2.2.1 Mô hình PRAM

Mô hình *Parallel Random Access Machine* (PRAM) gồm p bộ xử lý đồng bộ cùng truy cập một bộ nhớ chung; mỗi bước, mọi bộ xử lý thực hiện một lệnh. Biến thể EREW (Exclusive Read, Exclusive Write) cấm hai bộ xử lý cùng đọc hoặc cùng ghi một ô nhớ trong một bước. Blelloch phân tích các thuật toán scan trên EREW PRAM **blelloch1990**. PRAM bỏ qua chi phí đồng bộ, độ trễ và băng thông bộ nhớ – những yếu tố lại chiếm ưu thế trên phần cứng thật, như ta sẽ thấy ở **??**.

2.2.2 Work, span và định lý Brent

Một thuật toán song song được mô tả bằng một *đồ thị phụ thuộc* có hướng không chu trình (DAG), mỗi nút là một phép toán, mỗi cạnh là một phụ thuộc dữ liệu. Hai đại lượng đặc trưng:

- **Work** $\mathrm{W}(n)$: tổng số nút – thời gian chạy trên một bộ xử lý, T_1.
- **Span** (hay *depth*) $\mathrm{D}(n)$: độ dài đường đi dài nhất trong DAG – thời gian chạy khi có vô hạn bộ xử lý, T_∞.

Tỉ số W/D gọi là *độ song song* (parallelism) – số bộ xử lý tối đa có thể tận dụng hiệu quả. Với p bộ xử lý, hiển nhiên $T_p \ge \max(\mathrm{W}/p,\ \mathrm{D})$.

Định lý 2.1 (Brent **brent1974**). *Một thuật toán có work* W *và span* D *có thể được thực thi trên* p *bộ xử lý trong thời gian*

$$T_p \ \le \ \frac{\mathrm{W}}{p} + \mathrm{D}. \tag{2.1}$$

Định lý Brent cho thấy hai mục tiêu thiết kế: giảm D để có nhiều song song, và *không tăng* W so với thuật toán tuần tự tốt nhất.

Định nghĩa 2.3 (Work-efficient)**.** Thuật toán song song gọi là *work-efficient* nếu $\mathrm{W}(n) = \mathcal{O}(T_{\text{seq}}(n))$, trong đó T_{seq} là thời gian của thuật toán tuần tự tốt nhất.

Work-efficiency đặc biệt quan trọng khi $p \ll n$ – trường hợp luôn gặp trên CPU (p cỡ 10, n cỡ 10^8). Khi đó số hạng W/p trong (**??**) chiếm ưu thế tuyệt đối; một thuật toán có $\mathrm{W} = \mathcal{O}(n \log n)$ làm nhiều việc hơn bản tuần tự $\log_2 n$ lần (khoảng 27 lần với $n = 2^{27}$), nên cần ít nhất $\log_2 n$ bộ xử lý chỉ để hòa vốn. Đây là lý do thuật toán Hillis–Steele, dù có span tốt nhất, lại chậm nhất trong thực nghiệm (**??**).

2.2.3 Speedup và hiệu suất

Với T_{seq} là thời gian của *bản tuần tự tốt nhất* (không phải bản song song chạy 1 luồng) và T_p là thời gian với p luồng:

$$S_p = \frac{T_{\text{seq}}}{T_p}, \qquad E_p = \frac{S_p}{p}. \tag{2.2}$$

S_p là *speedup* (hệ số tăng tốc), E_p là *hiệu suất song song* (efficiency). Lý tưởng $S_p = p$, $E_p = 1$. Báo cáo luôn so sánh với bản tuần tự để tránh "thổi phồng" speedup: một thuật toán song song chậm khi chạy 1 luồng có thể có T_1/T_p đẹp nhưng vẫn chậm hơn bản tuần tự.

2.3 Giới hạn của tăng tốc

2.3.1 Định luật Amdahl và Gustafson

Nếu một phần f của chương trình buộc phải chạy tuần tự, định luật Amdahl **amdahl1967** cho

$$S_p \;\le\; \frac{1}{f + (1-f)/p} \;\xrightarrow{p\to\infty}\; \frac{1}{f}. \tag{2.3}$$

Với scan chia khối (**??**), phần tuần tự là việc quét p tổng khối, có chi phí $\mathcal{O}(p)$ so với $\mathcal{O}(n)$ của toàn bộ, tức $f \approx p/n$ – không đáng kể với n lớn. Như vậy theo Amdahl, scan gần như song song hóa hoàn toàn được. Định luật Gustafson **gustafson1988** xét bài toán có kích thước tăng theo p (weak scaling) và cho $S_p = p - f(p-1)$, lạc quan hơn nữa.

Tuy nhiên, cả hai định luật chỉ xét *số phép tính*. Như phần tiếp theo chỉ ra, với scan yếu tố quyết định lại là *lượng dữ liệu phải di chuyển*.

2.3.2 Mô hình roofline

Mô hình roofline **williams2009** đặc trưng một chương trình bằng *cường độ tính toán* (arithmetic intensity) I = số phép toán / số byte trao đổi với bộ nhớ chính, và chặn trên hiệu năng đạt được:

$$P \;\le\; \min\bigl(P_{\text{peak}},\; I \times \beta\bigr), \tag{2.4}$$

trong đó P_{peak} là hiệu năng tính toán đỉnh và β là băng thông bộ nhớ. Chương trình có I nhỏ nằm ở vùng "mái nghiêng" – *giới hạn bởi băng thông* (memory-bound): thêm nhân cũng không nhanh hơn khi băng thông đã bão hòa.

Với scan trên số nguyên 64 bit, mỗi phần tử cần ít nhất 1 phép cộng, đọc 8 byte và ghi 8 byte, nên

$$I_{\text{scan}} \le \frac{1}{16} \text{ phép cộng/byte} \approx 0{,}06.$$

Một nhân CPU hiện đại thực hiện được hàng tỉ phép cộng mỗi giây, trong khi băng thông DRAM của máy tính xách tay chỉ khoảng 15–60 GB/s. Tại $I = 0{,}06$, băng thông 15 GB/s chỉ "nuôi" được khoảng 10^9 phần tử/s – mức mà *một* nhân đã đạt tới. Hệ quả quan trọng:

> *Với dữ liệu lớn hơn bộ nhớ đệm, thời gian scan bị chặn dưới bởi $T \ge 16n/\beta$; speedup tối đa đạt được xấp xỉ $\beta_{\text{đa luồng}}/\beta_{\text{đơn luồng}}$, chứ không phải p.*

Để có mốc băng thông thực tế, báo cáo đo kernel *copy* song song ($y_i = x_i$, cũng 16 byte/phần tử, tương tự

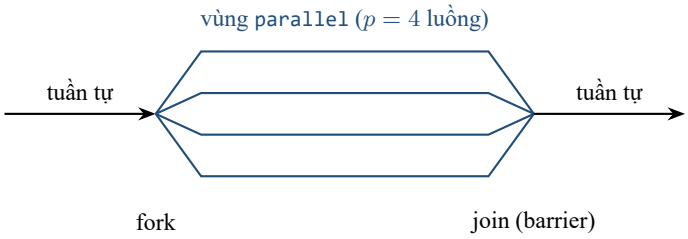
STREAM-copy **mccalpin1995**): đây là "trần" mà không thuật toán scan nào có thể vượt qua khi dữ liệu nằm ngoài cache.

2.4 Chuẩn lập trình OpenMP

OpenMP **openmp52**, **chapman2008** là chuẩn lập trình song song cho bộ nhớ chia sẻ, gồm các chỉ thị biên dịch (`#pragma omp`), hàm thư viện (`omp_*`) và biến môi trường (`OMP_*`). Ưu điểm lớn của OpenMP là cho phép song song hóa *tăng dần* một chương trình tuần tự: bỏ các chỉ thị đi, chương trình vẫn đúng.

2.4.1 Mô hình fork–join

Chương trình bắt đầu với một luồng chính. Gặp vùng `#pragma omp parallel`, luồng chính tạo (*fork*) một nhóm p luồng cùng thực thi khối lệnh; cuối vùng, các luồng đồng bộ và hợp lại (*join*) thành một (**??**). Thư viện `libgomp` giữ lại nhóm luồng giữa các vùng song song nên chi phí fork–join chỉ vài micro giây, nhưng vẫn đáng kể khi lặp lại nhiều lần với dữ liệu nhỏ.



Hình 2.1. Mô hình fork–join của OpenMP.

2.4.2 Các chỉ thị sử dụng trong đề tài

Bảng 2.1. Các cấu trúc OpenMP được dùng để cài đặt scan.

Cấu trúc	Ý nghĩa
`parallel`	Tạo nhóm luồng; mọi luồng thực thi khối lệnh.
`for schedule(static)`	Chia vòng lặp thành p đoạn liên tiếp bằng nhau; rào chắn ngầm định ở cuối.
`for … nowait`	Bỏ rào chắn ngầm định ở cuối vòng `for`.
`barrier`	Rào chắn tường minh: mọi luồng chờ nhau tại điểm này.
`single`	Chỉ một luồng thực thi khối lệnh; các luồng khác chờ ở cuối.
`simd`	Yêu cầu vector hóa vòng lặp (AVX2: 4 số int64 mỗi lệnh).
`reduction(+:s)`	Mỗi luồng có bản sao riêng của `s`, cộng dồn ở cuối.
`reduction(inscan,+:s)` + `scan inclusive(s)`	Chỉ thị scan chuẩn của OpenMP 5.0: trình biên dịch tự sinh thuật toán scan song song.
`omp_get_thread_num()`, `omp_get_num_threads()`	Chỉ số luồng và số luồng trong nhóm – dùng để tự chia dữ liệu theo khối.
`omp_get_wtime()`	Đồng hồ thời gian thực độ phân giải cao, dùng để đo hiệu năng.

2.4.3 Chỉ thị `scan` của OpenMP 5.0

Từ phiên bản 5.0 (2018), OpenMP hỗ trợ trực tiếp phép scan qua mệnh đề `reduction(inscan, op: var)` trên vòng lặp và chỉ thị `#pragma omp scan inclusive(var)` (hoặc `exclusive`), chia thân vòng lặp thành pha "cập nhật" trước chỉ thị và pha "sử dụng" sau chỉ thị (**??**). Lập trình viên chỉ khai báo ý định; việc chọn thuật toán thuộc về trình biên dịch.

Đoạn mã 2.1. Inclusive scan bằng chỉ thị chuẩn OpenMP 5.0.

```
elem_t s = 0;
#pragma omp parallel for reduction(inscan, +:s)
for (size_t i = 0; i < n; i++) {
    s += in[i];                 // pha ậcp ậnht
    #pragma omp scan inclusive(s)
    out[i] = s;                 // pha ửs ụdng
}
```

Chuẩn quy định một số ràng buộc: không được dùng `schedule` cùng `reduction(inscan)`, và thân vòng lặp có đúng một chỉ thị `scan`. Hiệu năng của cách tiếp cận này được đánh giá ở **??**.

2.5 Kiến trúc CPU đa nhân và các yếu tố ảnh hưởng hiệu năng

2.5.1 Phân cấp bộ nhớ

CPU hiện đại có nhiều cấp bộ nhớ đệm giữa nhân và DRAM: L1 (vài chục KiB, độ trễ khoảng 1 ns), L2 (vài trăm KiB đến vài MiB, khoảng 3–5 ns) và L3 dùng chung (vài MiB đến vài chục MiB, khoảng 10–20 ns), trong khi DRAM có độ trễ khoảng 80–100 ns **hennessy2019**. Dữ liệu được trao đổi theo *dòng cache* (cache line) 64 byte. Hai hệ quả với scan:

- **Truy cập tuần tự là tốt nhất**: bộ nạp trước (prefetcher) phần cứng nhận ra mẫu truy cập liên tiếp và nạp trước dữ liệu, che giấu độ trễ. Truy cập với bước nhảy lớn (như thuật toán Blelloch ở các tầng cao của cây) chỉ dùng 8 byte trong mỗi dòng 64 byte được nạp, lãng phí 7/8 băng thông.
- **Dữ liệu vừa cache thì nhanh hơn nhiều**: khi tổng kích thước mảng vào và ra nhỏ hơn L3, băng thông khả dụng cao hơn DRAM nhiều lần và mỗi nhân có L1/L2 riêng, nên speedup có thể gần tuyến tính.

2.5.2 False sharing

Khi hai luồng ghi vào hai biến *khác nhau* nằm trên cùng một dòng cache, giao thức nhất quán cache (MESI) buộc dòng đó liên tục bị chuyển qua lại giữa các nhân – hiện tượng *false sharing*. Trong scan chia khối, mảng `sums[p]` chứa tổng khối của các luồng là ứng viên điển hình: 8 phần tử int64 liên tiếp nằm cùng một dòng 64 byte. Cách khắc phục là *đệm* (padding) mỗi phần tử ra đủ 64 byte – vấn đề tương tự *bank conflict* trong shared memory của GPU, mà Harris và cộng sự giải quyết bằng macro `CONFLICT_FREE_OFFSET` **harris2007**.

2.5.3 CPU lai: P-core và E-core

Từ thế hệ thứ 12, Intel dùng kiến trúc lai: *P-core* hiệu năng cao (có Hyper-Threading, 2 luồng/nhân) và *E-core* tiết kiệm điện (1 luồng/nhân, xung nhịp và cache thấp hơn). CPU thực nghiệm Intel Core i5-1335U có 2 P-core và 8 E-core, tổng 12 luồng (**??**). Điều này ảnh hưởng tới song song hóa:

- **Mất cân bằng tải**: chia đều dữ liệu (`schedule(static)`) khiến luồng chạy trên E-core xong chậm hơn; tại mỗi barrier mọi luồng phải chờ luồng chậm nhất.
- **Hyper-Threading**: hai luồng trên cùng P-core chia sẻ đơn vị thực thi và cache L1/L2; với bài toán memory-bound, luồng thứ hai gần như không thêm băng thông.
- **Số luồng tối ưu** vì vậy thường nhỏ hơn số luồng logic.

2.5.4 First-touch và quản lý trang

Hệ điều hành cấp phát trang nhớ vật lý tại lần ghi *đầu tiên* (first-touch). Nếu đo ngay lần đầu dùng một mảng mới, kết quả sẽ gồm cả chi phí lỗi trang (page fault). Vì vậy mọi mảng trong thực nghiệm đều được khởi tạo song song trước khi đo, kèm vài lần chạy làm nóng (warm-up). Trên hệ thống NUMA nhiều socket, first-touch còn quyết định dữ liệu nằm ở nút nhớ nào; máy thực nghiệm chỉ có một socket nên yếu tố này không đáng kể.

2.6 Ứng dụng của scan

?? tóm tắt các ứng dụng tiêu biểu được Blelloch **blelloch1990** và Harris và cộng sự **harris2007** trình bày.

Bảng 2.2. Một số ứng dụng tiêu biểu của scan.

Ứng dụng	**Phép $\oplus$**	**Ý tưởng**
Stream compaction	$+$	Scan trên mảng cờ 0/1 cho ra vị trí đích của phần tử được giữ lại (**??**).
Radix sort	$+$	Mỗi lượt theo một bit: scan trên cờ bit để tính vị trí mới (phép *split*).
Quicksort song song	$+$	Phân hoạch quanh chốt bằng scan; dùng *segmented scan* cho nhiều đoạn cùng lúc.
Line-of-sight	max	Một điểm nhìn thấy được nếu góc của nó lớn hơn max-prescan của các góc phía trước.
Hệ truy hồi tuyến tính	hợp hàm affine	$x_i = a_i x_{i-1} + b_i$ giải bằng scan với phép hợp $(a,b) \circ (c,d) = (ac,\ bc+d)$.
Cộng số lớn	lan truyền nhớ	Lan truyền bit nhớ giữa các chữ số.
Summed-area table	$+$	Scan theo hàng rồi theo cột; lọc ảnh với cửa sổ bất kỳ trong $\mathcal{O}(1)$.
Cấp phát động	$+$	Mỗi luồng yêu cầu k_i phần tử; prescan cho ra địa chỉ bắt đầu.

CHƯƠNG 3

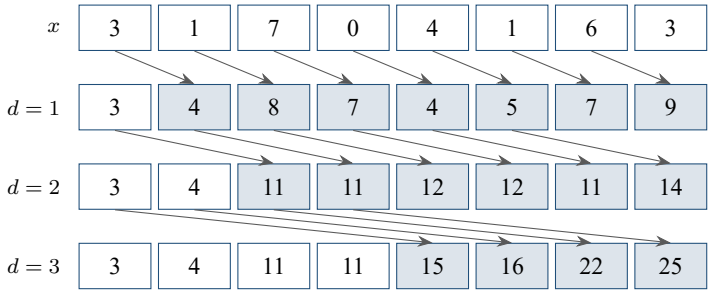
THUẬT TOÁN SONG SONG CHO TỔNG TIỀN TỐ

Chương này trình bày bốn cách tiếp cận song song hóa scan theo thứ tự lịch sử và theo mức độ "gần phần cứng": thuật toán Hillis–Steele (**??**), thuật toán work-efficient của Blelloch (**??**), scan chia khối dành cho trường hợp $n \gg p$ (**??**) và chỉ thị `scan` của OpenMP (**??**). Cuối chương so sánh các thuật toán trên cả phương diện lý thuyết lẫn lưu lượng bộ nhớ (**??**). Trong toàn chương, $\oplus$ là phép cộng và ví dụ minh họa dùng mảng $x = [3, 1, 7, 0, 4, 1, 6, 3]$ của Blelloch **blelloch1990**.

3.1 Thuật toán Hillis–Steele

3.1.1 Ý tưởng

Hillis và Steele **hillis1986** đề xuất thuật toán scan cho máy Connection Machine với hàng nghìn bộ xử lý. Ý tưởng dựa trên kỹ thuật *nhân đôi khoảng cách* (recursive doubling): ở bước $d = 1, 2, \ldots, \lceil \log_2 n \rceil$, mỗi phần tử $i \geq 2^{d-1}$ cộng thêm giá trị của phần tử cách nó 2^{d-1} vị trí về bên trái. Sau bước d, phần tử i chứa tổng của $\min(2^d, i+1)$ phần tử kết thúc tại i; sau $\lceil \log_2 n \rceil$ bước, nó chứa tổng của tất cả phần tử $x_0..x_i$, tức là inclusive scan (**??**).



Hình 3.1. Thuật toán Hillis–Steele trên 8 phần tử: 3 bước, mỗi bước phần tử i cộng thêm phần tử $i - 2^{d-1}$ (mũi tên). Ô tô màu là các phần tử được cập nhật. Hàng cuối là inclusive scan.

3.1.2 Mã giả và vấn đề cài đặt tại chỗ

Nếu cài đặt tại chỗ ($x[i] \mathrel{+}= x[i - 2^{d-1}]$) trên máy không đồng bộ tuyệt đối như CPU, một luồng có thể đọc $x[i - 2^{d-1}]$ *sau khi* luồng khác đã cập nhật nó trong cùng bước – một *tranh chấp dữ liệu* (data race) cho kết quả sai. Harris và cộng sự gặp đúng vấn đề này trên GPU khi các warp không chạy đồng thời và giải quyết bằng *bộ đệm kép* (double buffering) **harris2007**: đọc từ một mảng, ghi sang mảng kia, rồi hoán đổi vai trò (**??**). Giữa hai bước cần một rào chắn để đảm bảo mọi luồng đã ghi xong.

Thuật toán 2. Hillis–Steele với bộ đệm kép

Đầu vào: $x[0..n-1]$

Đầu ra: y = inclusive scan của x

$a \leftarrow x$; cấp phát mảng phụ $b[0..n-1]$;
for *off* $\leftarrow 1$; *off* $< n$; *off* $\leftarrow 2 \cdot$ *off* **do**
  for $i \leftarrow 0$ **to** $n-1$ **in parallel do**
    if $i \geq$ *off* **then** $b[i] \leftarrow a[i - \textit{off}] + a[i]$;
    else $b[i] \leftarrow a[i]$;
  barrier; hoán đổi $a \leftrightarrow b$;
return a;

3.1.3 Phân tích

- **Span:** $\mathrm{D}(n) = \lceil \log_2 n \rceil$ bước – tối ưu.
- **Work:** bước có *off* $= 2^{d-1}$ thực hiện $n - 2^{d-1}$ phép cộng, tổng cộng

$$\mathrm{W}(n) = \sum_{d=1}^{\log_2 n} \left(n - 2^{d-1}\right) = n \log_2 n - (n-1) = \mathcal{O}(n \log n).$$

- **Bộ nhớ:** mỗi bước đọc toàn bộ mảng a và ghi toàn bộ mảng b: $\log_2 n$ lượt quét, mỗi lượt khoảng $16n$ byte (với int64) – tức khoảng $16n \log_2 n$ byte so với $16n$ byte của bản tuần tự.

Thuật toán **không work-efficient**. Với $n = 2^{26}$, nó làm nhiều việc hơn bản tuần tự khoảng 25 lần; trên 12 luồng, kể cả khi song song hóa hoàn hảo, nó vẫn chậm hơn bản tuần tự khoảng hai lần. Thuật toán này hợp với phần cứng có số bộ xử lý xấp xỉ n (như một khối thread GPU xử lý vài trăm phần tử), nhưng không hợp với CPU.

3.2 Thuật toán work-efficient của Blelloch

3.2.1 Ý tưởng: hai lượt quét trên cây nhị phân cân bằng

Blelloch **blelloch1990** (dựa trên Ladner và Fischer **ladner1980**) quan sát rằng cây nhị phân dùng để tính *reduce* (tổng toàn mảng) đã chứa sẵn nhiều tổng riêng phần, và các tổng này đủ để dựng toàn bộ prefix sum. Thuật toán gồm hai pha, mỗi pha $\log_2 n$ bước, thực hiện *tại chỗ* trên mảng có kích thước $n = 2^k$ (nếu không thì đệm thêm phần tử đơn vị):

1. **Up-sweep (reduce):** đi từ lá lên gốc. Ở tầng d, mỗi nút cha (lưu tại vị trí con phải) nhận tổng của hai con. Kết thúc, $x[n-1]$ chứa tổng toàn mảng, các vị trí khác chứa tổng riêng phần của các cây con (**??**).
2. **Down-sweep:** gán phần tử đơn vị (0) cho gốc rồi đi từ gốc xuống lá. Ở mỗi nút, con trái nhận giá trị của cha; con phải nhận giá trị của cha cộng với giá trị cũ của con trái (tổng của cây con trái từ pha up-sweep). Kết thúc, mảng chứa *exclusive* scan (**??**).

Thuật toán 3. Blelloch scan (exclusive, tại chỗ, $n = 2^k$)

Đầu vào: $x[0..n-1]$

Đầu ra: x = exclusive scan của đầu vào

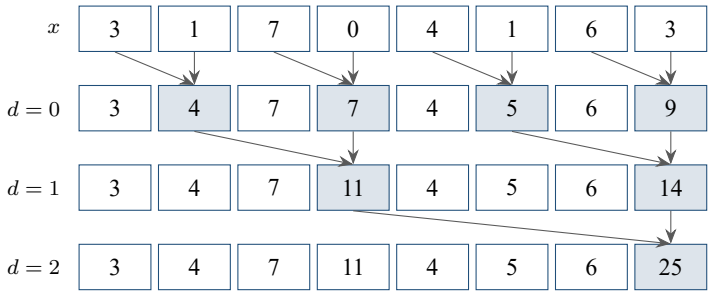
```
// Up-sweep
```

for $d \leftarrow 0$ **to** $\log_2 n - 1$ **do**
  for $k \leftarrow 0$ **to** $n-1$ ***bước*** 2^{d+1} **in parallel do**
    $x[k + 2^{d+1} - 1] \leftarrow x[k + 2^d - 1] + x[k + 2^{d+1} - 1]$;

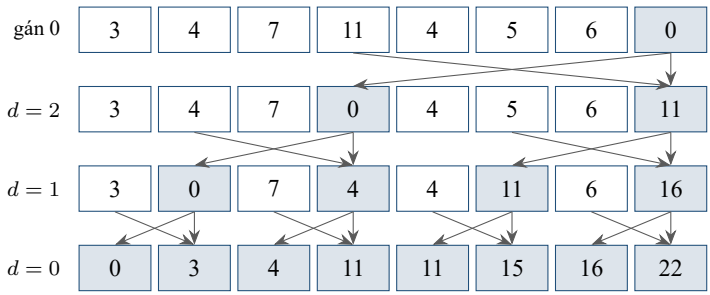
```
// Down-sweep
```

$x[n-1] \leftarrow 0$;

for $d \leftarrow \log_2 n - 1$ **downto** 0 **do**
  for $k \leftarrow 0$ **to** $n-1$ ***bước*** 2^{d+1} **in parallel do**
    $t \leftarrow x[k + 2^d - 1]$;
    $x[k + 2^d - 1] \leftarrow x[k + 2^{d+1} - 1]$;
    $x[k + 2^{d+1} - 1] \leftarrow t + x[k + 2^{d+1} - 1]$;



Hình 3.2. Pha up-sweep: ở tầng d, phần tử $k + 2^{d+1} - 1$ cộng thêm phần tử $k + 2^d - 1$. Sau $\log_2 n = 3$ bước, $x[7] = 25$ là tổng toàn mảng.



Hình 3.3. Pha down-sweep: con trái nhận giá trị của cha, con phải nhận cha cộng con trái cũ. Hàng cuối là exclusive scan $[0, 3, 4, 11, 11, 15, 16, 22]$, khớp với ví dụ của Blelloch **blelloch1990**.

3.2.2 Tính đúng đắn

Định lý 3.1 (Blelloch **blelloch1990**, Định lý 1.1)**.** *Sau pha down-sweep, mỗi nút của cây chứa tổng của tất cả các lá đứng trước nó theo thứ tự duyệt tiền thứ tự (preorder). Đặc biệt, lá thứ i chứa $x_0 + \cdots + x_{i-1}$.*

Chứng minh. Quy nạp từ gốc xuống. Gốc không có lá nào đứng trước nên giá trị đúng là phần tử đơn vị 0 – đúng như được gán. Giả sử nút cha v có giá trị đúng, tức bằng tổng các lá thuộc vùng A (các lá đứng trước v).

- *Con trái* của v có đúng tập lá đứng trước là A, vì trong duyệt tiền thứ tự con trái được thăm ngay sau v. Do đó gán giá trị của cha cho con trái là đúng.
- *Con phải* có tập lá đứng trước là $A \cup B$, với B là các lá của cây con trái. Tổng các lá của B chính là giá trị con trái sau up-sweep (được giữ trong biến t). Vì vậy con phải nhận giá trị cha $+ t$ là đúng.

Theo quy nạp, mọi nút, trong đó có các lá, đều có giá trị đúng. Thứ tự tiền thứ tự trên các lá trùng với thứ tự trái–phải của mảng, nên lá i chứa $\sum_{j<i} x_j$. □

3.2.3 Phân tích

- **Work:** up-sweep thực hiện $n/2 + n/4 + \cdots + 1 = n - 1$ phép cộng; down-sweep thực hiện $n - 1$ phép cộng và $n - 1$ phép hoán đổi. Tổng $\mathrm{W}(n) = \mathcal{O}(n)$: thuật toán **work-efficient harris2007**.
- **Span:** $\mathrm{D}(n) = 2\log_2 n$ – gấp đôi Hillis–Steele, vẫn là $\mathcal{O}(\log n)$.
- **Chuyển sang inclusive:** $y_i = \mathrm{prescan}_i + x_i$, thêm một lượt song song.

Nhược điểm trên CPU. Dù tối ưu về work, thuật toán có ba điểm yếu khi chạy trên CPU với dữ liệu lớn:

1. **Truy cập bộ nhớ có bước nhảy 2^{d+1}:** từ tầng $d = 3$ trở lên, mỗi phép cộng chạm vào một dòng cache khác nhau, chỉ dùng 8/64 byte. Các tầng thấp đọc lại toàn bộ mảng nhiều lần, nên tổng lưu lượng bộ nhớ lớn hơn nhiều so với $16n$ byte.
2. **Nhiều rào chắn:** $2\log_2 n \approx 52$ barrier với $n = 2^{26}$; ở các tầng trên cùng chỉ còn vài phần tử, đa số luồng ngồi chờ.
3. **Đệm lên lũy thừa của 2:** với $n = 2^k + 1$, kích thước làm việc gần gấp đôi.

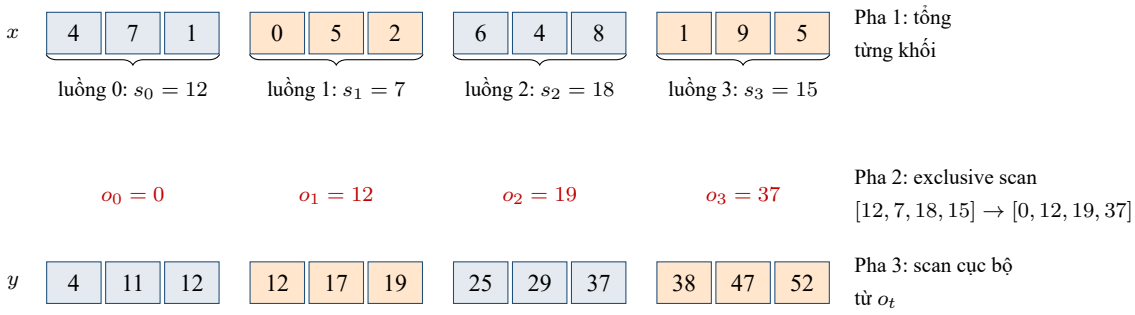
3.3 Scan chia khối cho trường hợp $n \gg p$

3.3.1 Ý tưởng

Cả hai thuật toán trên đều giả định có $\mathcal{O}(n)$ bộ xử lý. Khi chỉ có $p \ll n$ bộ xử lý, Blelloch **blelloch1990** đề xuất chia mảng thành p khối liên tiếp, mỗi khối $\lceil n/p \rceil$ phần tử, và thực hiện ba pha (**??**):

1. **Pha 1 – reduce cục bộ:** mỗi bộ xử lý t tính *tuần tự* tổng s_t của khối mình.
2. **Pha 2 – scan các tổng khối:** tính exclusive scan của $[s_0, \ldots, s_{p-1}]$, được $o_t = s_0 + \cdots + s_{t-1}$ – độ lệch (offset) của khối t. Vì p nhỏ, bước này làm tuần tự ($\mathcal{O}(p)$) hoặc bằng cây ($\mathcal{O}(\log p)$).
3. **Pha 3 – scan cục bộ:** mỗi bộ xử lý scan *tuần tự* khối của mình, bắt đầu từ o_t.

Đây cũng là cách Harris và cộng sự mở rộng scan GPU cho mảng kích thước bất kỳ: scan từng khối, ghi tổng mỗi khối vào mảng `SUMS`, scan `SUMS` thành `INCR`, rồi cộng đều `INCR[i]` vào khối i **harris2007**.



Hình 3.4. Scan chia khối với $n = 12$, $p = 4$ (ví dụ theo **blelloch1990**, ở dạng inclusive). Pha 1 và pha 3 hoàn toàn song song; pha 2 chỉ xử lý p số.

3.3.2 Phân tích

Theo Blelloch **blelloch1990**, thời gian trên EREW PRAM là

$$T_S(n,p) = 2\big(\lceil n/p \rceil + \lceil \log_2 p \rceil\big) = \mathcal{O}(n/p + \log p), \tag{3.1}$$

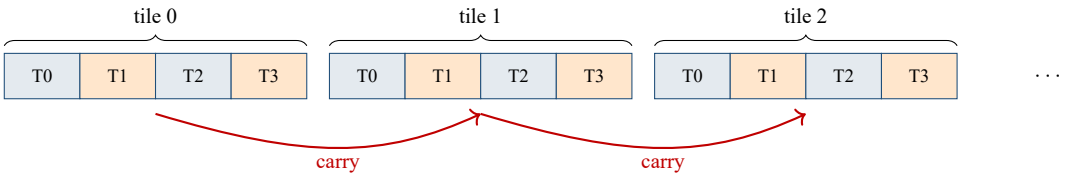
đạt speedup tối ưu khi $n/p \geq \log p$ – điều kiện luôn thỏa trên CPU. Work là $2n + \mathcal{O}(p)$: khoảng gấp đôi bản tuần tự (mỗi phần tử được cộng ở pha 1 và pha 3), vẫn $\mathcal{O}(n)$. Chỉ có **hai** rào chắn, và mọi truy cập bộ nhớ đều **liên tiếp** – lý tưởng cho prefetcher.

Lưu lượng bộ nhớ. Pha 1 đọc x ($8n$ byte), pha 3 đọc lại x và ghi y ($16n$ byte): tổng $24n$ byte, gấp 1,5 lần mức tối thiểu $16n$. Với dữ liệu lớn hơn cache, thời gian bị chặn dưới bởi $24n/\beta$: ngay cả song song hoàn hảo, scan chia khối cơ bản cũng chậm hơn kernel copy khoảng 1,5 lần.

Biến thể scan-then-propagate. Một cách sắp xếp khác: pha 1 scan cục bộ luôn (ghi y), pha 3 cộng offset vào y. Cách này đọc x một lần nhưng đọc–ghi y hai lần: $8n + 8n + 16n = 32n$ byte, tệ hơn. Báo cáo dùng biến thể reduce-then-scan.

3.3.3 Cải tiến: chia tile vừa bộ nhớ đệm

Để loại bỏ lượt đọc thừa, nhóm áp dụng ý tưởng *chia tile* (tinh thần giống kỹ thuật mỗi luồng xử lý nhiều phần tử của Harris và cộng sự **harris2007**): thay vì p khối lớn n/p, mảng được chia thành các *tile*, mỗi tile gồm p khối nhỏ B phần tử, sao cho $8B$ byte vừa với cache L2 của một nhân (**??**). Ba pha được thực hiện lần lượt cho từng tile; biến *carry* mang tổng các tile trước sang tile sau. Pha 3 đọc lại khối *ngay sau* pha 1, khi dữ liệu còn trong L2, nên mảng x chỉ đọc từ DRAM một lần: lưu lượng giảm từ $24n$ xuống $16n$ byte – đúng mức tối thiểu.



Hình 3.5. Blocked-opt: mảng chia thành các tile gồm p khối B phần tử (ở đây $p = 4$, luồng Tt xử lý khối thứ t của mỗi tile). Ba pha thực hiện trong từng tile; tổng của tile được cộng dồn vào *carry* cho tile kế tiếp.

Cái giá phải trả là số rào chắn tăng từ 2 lên $\lceil n/(pB) \rceil$ (một rào chắn mỗi tile). B vì vậy là tham số đánh đổi:

B nhỏ thì nhiều rào chắn, B lớn thì khối tràn khỏi L2. Ảnh hưởng của B được khảo sát ở **??**.

3.4 Chỉ thị `scan` của OpenMP

Với chỉ thị `reduction(inscan,+)` (**??**), lập trình viên không chọn thuật toán. GCC/libgomp cài đặt chỉ thị này theo sơ đồ chia khối hai lượt: mỗi luồng chạy pha "cập nhật" trên đoạn của mình, lưu giá trị cục bộ của biến thu gọn cho *từng* lần lặp vào bộ đệm tạm, sau đó các tổng đoạn được scan và mỗi luồng chạy pha "sử dụng" với giá trị đã hiệu chỉnh. Bộ đệm tạm có kích thước $\mathcal{O}(n)$ làm tăng lưu lượng bộ nhớ, và thân vòng lặp bị tách thành hai bản sao nên khó vector hóa. Ưu điểm là ngắn gọn, di động; nhược điểm là hiệu năng phụ thuộc hoàn toàn vào cài đặt của trình biên dịch.

3.5 So sánh các thuật toán

Bảng 3.1. So sánh lý thuyết các thuật toán scan (n phần tử int64, p luồng, B kích thước khối). Lưu lượng DRAM tính cho dữ liệu lớn hơn cache.

Thuật toán	Work	Span	Barrier	Bộ nhớ phụ	Lưu lượng DRAM
Tuần tự	n	n	0	0	$16n$
Hillis–Steele	$n \log_2 n$	$\log_2 n$	$\log_2 n$	n	$\approx 16n \log_2 n$
Blelloch	$2n$	$2 \log_2 n$	$2 \log_2 n$	$\leq 2n$	$\gg 16n$ (bước nhảy)
Blocked	$2n$	$2n/p + p$	2	p	$24n$
Blocked-opt	$2n$	$2n/p + n/B$	$n/(pB)$	$2p$	$16n$
`omp scan`	$\approx 2n$	$\approx 2n/p$	tùy cài đặt	$\mathcal{O}(n)$	$> 24n$

?? gợi ý trước kết quả thực nghiệm: trên CPU, work và span không đủ để dự đoán hiệu năng; *lưu lượng bộ nhớ* mới là đại lượng quyết định khi dữ liệu lớn. Hillis–Steele và Blelloch – hai thuật toán có span $\mathcal{O}(\log n)$ đẹp nhất – lại có lưu lượng lớn nhất. Ngược lại, Blocked-opt có span "kém" nhất nhưng lưu lượng tối thiểu.

CHƯƠNG 4

CÀI ĐẶT BẰNG OPENMP

Chương này mô tả cách các thuật toán ở **??** được hiện thực bằng C11 và OpenMP: cấu trúc mã nguồn (**??**), chi tiết từng phiên bản (**??**), các kỹ thuật tối ưu (**??**), kiểm thử (**??**) và công cụ đo hiệu năng (**??**). Toàn bộ mã nguồn có trong **??** và trên kho GitHub của nhóm.

4.1 Kiến trúc mã nguồn

Bảng 4.1. Các tệp mã nguồn.

Tệp	Nội dung
`src/scan.h`	Giao diện chung: kiểu `elem_t`, cấu trúc vùng làm việc `scan_ws`, nguyên mẫu các hàm scan, bảng đăng ký thuật toán.
`src/scan.c`	Cài đặt sáu phiên bản scan và kernel copy tham chiếu.
`src/test_scan.c`	Bộ kiểm thử tính đúng: so sánh mọi phiên bản với bản tuần tự.
`src/bench.c`	Chương trình đo hiệu năng, xuất CSV.
`src/compact.c`	Ứng dụng stream compaction dựa trên scan.
`scripts/run_all.sh`	Chạy toàn bộ thực nghiệm, ghi thông tin máy và kết quả thô.
`scripts/plots.py`	Tổng hợp trung vị, vẽ biểu đồ và sinh bảng LaTeX.
`Makefile`	Biên dịch với `gcc -O3 -march=native -fopenmp`.

Giao diện thống nhất. Mọi phiên bản có cùng chữ ký (**??**), cho phép chương trình đo và bộ kiểm thử duyệt qua một bảng các con trỏ hàm mà không cần mã riêng cho từng thuật toán. Bộ nhớ phụ (buffer cho Hillis–Steele và Blelloch) được cấp phát *trước* khi đo qua `scan_ws_create`, để thời gian đo không bao gồm chi phí `malloc` và lỗi trang.

Đoạn mã 4.1. Giao diện chung trong `scan.h` (rút gọn).

```
typedef int64_t elem_t;
typedef struct {
    elem_t *tmp;      // buffer ụph >= next_pow2(n) ầphn ửt
    size_t  tmp_cap;
    size_t  block;    // kích ướthc ốkhi B cho blocked_opt
} scan_ws;

typedef void (*scan_fn)(const elem_t *in, elem_t *out, size_t n, scan_ws *ws);

void scan_seq          (const elem_t *in, elem_t *out, size_t n, scan_ws *ws);
void scan_hillis_steele(const elem_t *in, elem_t *out, size_t n, scan_ws *ws);
void scan_blelloch     (const elem_t *in, elem_t *out, size_t n, scan_ws *ws);
void scan_blocked      (const elem_t *in, elem_t *out, size_t n, scan_ws *ws);
```

```
void scan_blocked_opt  (const elem_t *in, elem_t *out, size_t n, scan_ws *ws);
void scan_omp          (const elem_t *in, elem_t *out, size_t n, scan_ws *ws);
```

Cấp phát bộ nhớ. Mọi mảng được cấp phát bằng `aligned_alloc(64, …)`, căn lề theo dòng cache, để các khối của luồng bắt đầu đúng biên dòng cache và trình biên dịch có thể dùng lệnh nạp vector căn lề.

4.2 Cài đặt các phiên bản

4.2.1 Hillis–Steele

Cài đặt (**??**) mở *một* vùng `parallel` duy nhất bao trọn mọi bước, tránh chi phí fork–join $\log_2 n$ lần. Mỗi bước tách làm hai vòng `for`: vòng $i < off$ chỉ sao chép, vòng $i \geq off$ thực hiện phép cộng. Việc tách bỏ nhánh `if` trong thân vòng lặp giúp trình biên dịch vector hóa được. Vòng thứ nhất dùng `nowait` vì hai vòng ghi vào hai vùng rời nhau của `b`. Việc hoán đổi con trỏ đặt trong `single`: mọi luồng thấy cùng giá trị `a`, `b` vì đây là biến dùng chung khai báo ngoài vùng song song, và rào chắn ngầm định cuối `single` đảm bảo không luồng nào bắt đầu bước sau khi hoán đổi chưa xong.

Đoạn mã 4.2. Hillis–Steele với bộ đệm kép.

```
elem_t *a = out, *b = ws->tmp;
#pragma omp parallel
{
    #pragma omp for schedule(static)
    for (size_t i = 0; i < n; i++) a[i] = in[i];

    for (size_t off = 1; off < n; off <<= 1) {
        #pragma omp for schedule(static) nowait
        for (size_t i = 0; i < off; i++) b[i] = a[i];
        #pragma omp for schedule(static)
        for (size_t i = off; i < n; i++) b[i] = a[i - off] + a[i];
        #pragma omp single
        { elem_t *t = a; a = b; b = t; }      // barrier ẩngm định
    }
    if (a != out) {
        #pragma omp for schedule(static)
        for (size_t i = 0; i < n; i++) out[i] = a[i];
    }
}
```

CHƯƠNG 5

THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1 Môi trường

5.2 Strong

5.3 Block

5.4 App

CHƯƠNG 6

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

PHỤ LỤC A

MÃ NGUỒN CHÍNH

Phụ lục này trích toàn văn tệp `src/scan.c` – cài đặt sáu phiên bản scan và kernel tham chiếu. Các tệp còn lại (`bench.c`, `test_scan.c`, `compact.c`, script) có trên kho GitHub của nhóm.

Đoạn mã A.1. Tệp `src/scan.c`.

```c
/*
 * scan.c - Cài đặt các phiên bản Parallel Prefix Sum bằng OpenMP.
 *
 *   seq            : tuần tự, O(n)                                  (baseline)
 *   hillis_steele  : Hillis & Steele 1986, double buffer, O(n log n)  (không work-efficient)
 *   blelloch       : Blelloch 1990, up-sweep/down-sweep giữ cạch, O(n)
 *   blocked        : chia khối 3 pha (reduce - scan tổng khối - scan lục cộb), O(n)
 *   blocked_opt    : blocked + chia tile vừa cache L2 + chống false sharing + SIMD
 *   omp_scan       : tích hợp sẵn OpenMP 5.0  reduction(inscan, +)
 */
#include "scan.h"

#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* ------------------------------------------------------------------ */
/* Tiện ích                                                           */
/* ------------------------------------------------------------------ */
size_t next_pow2(size_t n)
{
    size_t m = 1;
    while (m < n) m <<= 1;
    return m;
}

elem_t *alloc_elems(size_t n)
{
    size_t bytes = (n ? n : 1) * sizeof(elem_t);
    bytes = (bytes + CACHE_LINE - 1) / CACHE_LINE * CACHE_LINE;  /* aligned_alloc yêu cầu bội số */
    elem_t *p = aligned_alloc(CACHE_LINE, bytes);
    if (!p) {
        fprintf(stderr, "Khong du bo nho (%zu bytes)\n", bytes);
        exit(EXIT_FAILURE);
    }
    return p;
}

scan_ws scan_ws_create(size_t n, size_t block)
{
    scan_ws ws;
    ws.tmp_cap = next_pow2(n ? n : 1);
    ws.tmp     = alloc_elems(ws.tmp_cap);
    ws.block   = block ? block : DEFAULT_BLOCK;
    /* first-touch song song: ánh xạ trang ướtrc khi đo */
    #pragma omp parallel for schedule(static)
    for (size_t i = 0; i < ws.tmp_cap; i++) ws.tmp[i] = 0;
    return ws;
}

void scan_ws_destroy(scan_ws *ws)
{
    free(ws->tmp);
```

```
    ws->tmp = NULL;
    ws->tmp_cap = 0;
}

/* ---------------------------------------------------------------------- */
/* 1. àTun ựt                                                              */
/* ---------------------------------------------------------------------- */
void scan_seq(const elem_t *in, elem_t *out, size_t n, scan_ws *ws)
{
    (void)ws;
    elem_t s = 0;
    for (size_t i = 0; i < n; i++) {
        s += in[i];
        out[i] = s;
    }
}

/* ---------------------------------------------------------------------- */
/* 2. Hillis-Steele: log2(n) ướbc, ỗmi ướbc ộcng àphn ửt cách off       */
/* à   Cn double buffer vì các òlung không ạchy đồng ộb (lockstep).    */
/* ---------------------------------------------------------------------- */
void scan_hillis_steele(const elem_t *in, elem_t *out, size_t n, scan_ws *ws)
{
    if (n == 0) return;
    elem_t *a = out;      /* buffer đọc  */
    elem_t *b = ws->tmp;  /* buffer ghi  */

    #pragma omp parallel
    {
        #pragma omp for schedule(static)
        for (size_t i = 0; i < n; i++) a[i] = in[i];

        for (size_t off = 1; off < n; off <<= 1) {
            /* Hai vòng ặlp tách riêng để tránh ẽr nhánh -> trình biên ịdch vector hóa được */
            #pragma omp for schedule(static) nowait
            for (size_t i = 0; i < off; i++) b[i] = a[i];
            #pragma omp for schedule(static)
            for (size_t i = off; i < n; i++) b[i] = a[i - off] + a[i];
            /* rào ắchn àngm định ở cuối vòng for ở trên */
            #pragma omp single
            { elem_t *t = a; a = b; b = t; }   /* hoán đổi buffer; single có rào ắchn àngm định */
        }

        if (a != out) {
            #pragma omp for schedule(static)
            for (size_t i = 0; i < n; i++) out[i] = a[i];
        }
    }
}

/* ---------------------------------------------------------------------- */
/* 3. Blelloch: up-sweep (reduce) + down-sweep trên cây ịnh phân ẩn.   */
/*    Làm ạti ỗch trên buffer kích ướthc m = next_pow2(n) (pad ằbng 0). */
/* ế   Kt àqu ựt nhiên là exclusive scan; ộcng in[i] để ra inclusive.  */
/* ---------------------------------------------------------------------- */
void scan_blelloch(const elem_t *in, elem_t *out, size_t n, scan_ws *ws)
{
    if (n == 0) return;
    const size_t m = next_pow2(n);
    elem_t *x = ws->tmp;

    #pragma omp parallel
    {
        #pragma omp for schedule(static)
        for (size_t i = 0; i < m; i++) x[i] = (i < n) ? in[i] : 0;

        /* Up-sweep: stride = 2^(d+1) */
```

```
        for (size_t stride = 2; stride <= m; stride <<= 1) {
            const size_t half = stride >> 1;
            #pragma omp for schedule(static)
            for (size_t k = stride - 1; k < m; k += stride)
                x[k] += x[k - half];
        }

        #pragma omp single
        x[m - 1] = 0;                       /* gán ầphn ừt đơn ịv cho ốgc */

        /* Down-sweep */
        for (size_t stride = m; stride >= 2; stride >>= 1) {
            const size_t half = stride >> 1;
            #pragma omp for schedule(static)
            for (size_t k = stride - 1; k < m; k += stride) {
                elem_t t    = x[k - half];  /* giá ịtr con trái ừt up-sweep   */
                x[k - half] = x[k];         /* con trái  <- giá ịtr ủca cha    */
                x[k]       += t;            /* con ảphi  <- cha + con trái ũc  */
            }
        }

        /* exclusive -> inclusive */
        #pragma omp for schedule(static)
        for (size_t i = 0; i < n; i++) out[i] = x[i] + in[i];
    }
}

/* ------------------------------------------------------------------ */
/* 4. Blocked (reduce-then-scan), ỗmi ồlung ộmt ốkhi liên ếtip n/p:    */
/*    Pha 1: ỗmi ồlung tính ổtng ốkhi ủca mình                         */
/*    Pha 2: ộmt ồlung scan ảmng p ổtng ốkhi -> offset                  */
/*    Pha 3: ỗmi ồlung scan ầtun ựt ốkhi ủca mình, ắbt đầu ừt offset    */
/* ------------------------------------------------------------------ */
#define MAX_THREADS 256

void scan_blocked(const elem_t *in, elem_t *out, size_t n, scan_ws *ws)
{
    (void)ws;
    if (n == 0) return;
    elem_t sums[MAX_THREADS + 1];

    #pragma omp parallel
    {
        const int    p   = omp_get_num_threads();
        const int    t   = omp_get_thread_num();
        const size_t len = (n + p - 1) / p;
        const size_t lo  = (size_t)t * len < n ? (size_t)t * len : n;
        const size_t hi  = lo + len < n ? lo + len : n;

        /* Pha 1 */
        elem_t s = 0;
        for (size_t i = lo; i < hi; i++) s += in[i];
        sums[t + 1] = s;
        #pragma omp barrier

        /* Pha 2 */
        #pragma omp single
        {
            sums[0] = 0;
            for (int i = 1; i <= p; i++) sums[i] += sums[i - 1];
        }   /* rào ắchn ầngm định */

        /* Pha 3 */
        s = sums[t];
        for (size_t i = lo; i < hi; i++) {
            s += in[i];
            out[i] = s;
```

```
        }
    }
}

/* ---------------------------------------------------------------------- */
/* 5. Blocked ối ưu:                                                      */
/*  - Chia ảmng thành các TILE ồgm p ốkhi, ỗmi ốkhi B ầphn ừt (ừva L2).  */
/*    Pha 1 đọc ốkhi ừt RAM vào cache, pha 3 đọc ại NGAY ừt cache       */
/*    => ảmng vào ịch đọc ừt RAM 1 ầln (16 B/ầphn ừt thay vì 24 B).     */
/*  - ảMng ổtng ốkhi được pad 64 byte (ốchng false sharing),             */
/*    double-buffer theo tile => ịch 1 barrier ỗmi tile.                */
/*  - ỗMi ồlung ựt tính offset ủca mình (p ỏnh) => không ầcn single.    */
/*  - Pha 1 dùng omp simd reduction (vector hóa AVX2).                   */
/* ---------------------------------------------------------------------- */
typedef struct {
    elem_t v;
    char   pad[CACHE_LINE - sizeof(elem_t)];
} padded_elem;

void scan_blocked_opt(const elem_t *in, elem_t *out, size_t n, scan_ws *ws)
{
    if (n == 0) return;
    const size_t B = ws->block;
    static padded_elem part[2][MAX_THREADS];

    #pragma omp parallel
    {
        const int    p     = omp_get_num_threads();
        const int    t     = omp_get_thread_num();
        const size_t tile  = B * (size_t)p;
        elem_t       carry = 0;            /* ổtng ọmi tile ướtrc (ỗmi ồlung ữgi 1 ảbn) */
        int          buf   = 0;

        for (size_t base = 0; base < n; base += tile) {
            const size_t lo = base + (size_t)t * B < n ? base + (size_t)t * B : n;
            const size_t hi = lo + B < n ? lo + B : n;

            /* Pha 1: ổtng ốkhi (vector hóa) */
            elem_t s = 0;
            #pragma omp simd reduction(+:s)
            for (size_t i = lo; i < hi; i++) s += in[i];
            part[buf][t].v = s;

            #pragma omp barrier

            /* Pha 2: offset = carry + ổtng các ốkhi đứng ướtrc; ậcp ậnht carry */
            elem_t off = carry, tot = 0;
            for (int j = 0; j < p; j++) {
                const elem_t v = part[buf][j].v;
                if (j < t) off += v;
                tot += v;
            }
            carry += tot;

            /* Pha 3: scan ựcc ộb, ữd ệliu đang ằnm trong L2 */
            s = off;
            for (size_t i = lo; i < hi; i++) {
                s += in[i];
                out[i] = s;
            }
            buf ^= 1;   /* tile ếk ếtip ghi sang buffer kia: không ầcn barrier ứth hai */
        }
    }
}

/* ---------------------------------------------------------------------- */
/* 6. ịCh ịth ẩchun OpenMP 5.0: reduction(inscan, +) + #pragma omp scan */
```

```c
/* ---------------------------------------------------------------- */
void scan_omp(const elem_t *in, elem_t *out, size_t n, scan_ws *ws)
{
    (void)ws;
    if (n == 0) return;   /* libgomp (GCC 15) ịb ỗli ộb ớnh khi vòng inscan ỗrng và p > 1 */
    elem_t s = 0;
    /* ưLu ý: ẩchun OpenMP không cho phép ệmnh đề schedule đi cùng reduction(inscan).
     * Không dùng "for simd": GCC ấcp phát ảmng ạtm ỡc n trên stack -> tràn stack ớvi n ớln. */
    #pragma omp parallel for reduction(inscan, +:s)
    for (size_t i = 0; i < n; i++) {
        s += in[i];
        #pragma omp scan inclusive(s)
        out[i] = s;
    }
}

/* ---------------------------------------------------------------- */
/* Kernel tham ếchiu: sao chép song song đọ(c n + ghi n)              */
/* Dùng đểđ o ảbng thông ộb ớnh ựthc ết đạt được (STREAM-copy).       */
/* ---------------------------------------------------------------- */
void kernel_copy(const elem_t *in, elem_t *out, size_t n, scan_ws *ws)
{
    (void)ws;
    #pragma omp parallel for simd schedule(static)
    for (size_t i = 0; i < n; i++) out[i] = in[i];
}

/* ---------------------------------------------------------------- */
const scan_algo SCAN_ALGOS[] = {
    {"seq",           scan_seq,           1},
    {"hillis_steele", scan_hillis_steele, 1},
    {"blelloch",      scan_blelloch,      1},
    {"blocked",       scan_blocked,       1},
    {"blocked_opt",   scan_blocked_opt,   1},
    {"omp_scan",      scan_omp,           1},
    {"copy",          kernel_copy,        0},
};
const int SCAN_NALGOS = (int)(sizeof(SCAN_ALGOS) / sizeof(SCAN_ALGOS[0]));

const scan_algo *scan_find(const char *name)
{
    for (int i = 0; i < SCAN_NALGOS; i++)
        if (strcmp(SCAN_ALGOS[i].name, name) == 0) return &SCAN_ALGOS[i];
    return NULL;
}
```

PHỤ LỤC B

SỐ LIỆU THỰC NGHIỆM